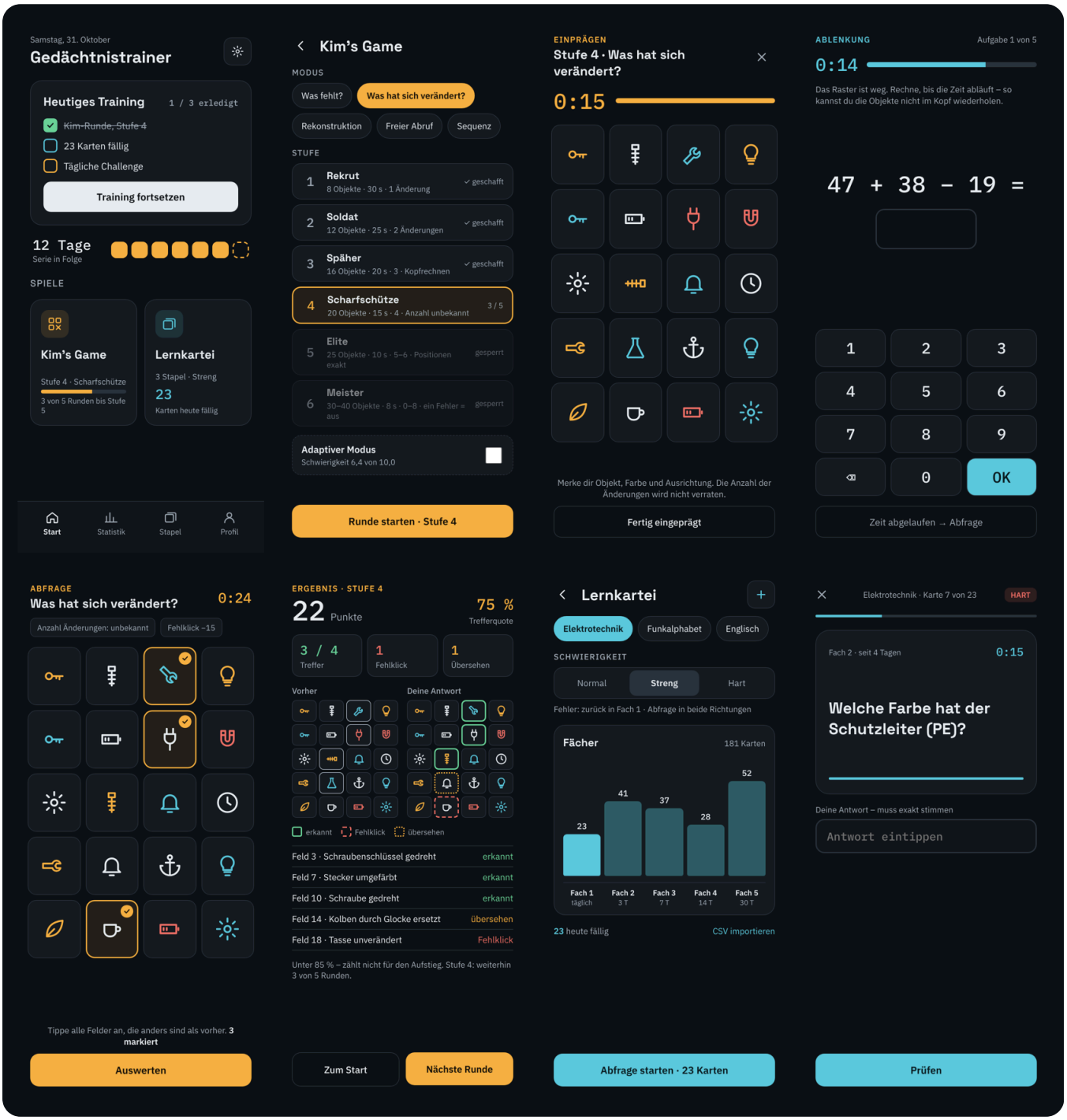
SPEZIFIKATION FÜR CLAUDE CODE · STAND 31.10.2026

Gedächtnistrainer Android-App mit Kim's Game und Lernkartei

Native Android-App (Kotlin, Jetpack Compose), komplett offline. Zwei Spiele in einer App: **Kim's Game** trainiert das visuelle Kurzzeitgedächtnis in sechs harten Stufen, die **Lernkartei** nach dem Leitner-System trainiert das Langzeitgedächtnis. Dieses Dokument enthält alles, was zum Bauen nötig ist: Spielregeln mit genauen Zahlen, Architektur, Datenmodell, Design-Vorgaben und alle 8 Bildschirme.



0 · So arbeitest du mit diesem Dokument in Claude Code

1. Neuen Ordner anlegen, diese PDF als `docs/spec.pdf` hineinlegen, Claude Code im Ordner starten.
2. Phase für Phase bauen lassen (Kapitel 8). Jede Phase endet mit einer lauffähigen Version und einem klaren Abnahmekriterium – erst dann die nächste Phase starten.
3. Die Zahlen in Kapitel 2 und 3 sind verbindlich. Sie gehören in eine JSON-Datei (`app/src/main/assets/balancing.json`), nicht hart in den Code.
4. Die Screenshots in Kapitel 7 sind die Designvorlage. Farben, Schriften und Abstände stehen exakt in Kapitel 6.

Start-Prompt für Claude Code

```
Lies docs/spec.pdf vollständig. Lege eine CLAUDE.md an, die die
wichtigsten Regeln daraus zusammenfasst (Stack, Paketstruktur,
Design-Tokens, Balancing in assets/balancing.json, Tests für
die Engines). Setze dann Phase 1 aus Kapitel 8 um. Halte dich
genau an die Zahlen aus Kapitel 2 und das Design aus Kapitel 6/7.
Schreib Unit-Tests für die Engine und lass sie laufen, bevor du
mir Bescheid gibst.
```

Folge-Prompts

```
Phase 1 ist abgenommen. Setze jetzt Phase 2 aus docs/spec.pdf um.
```

Feste Regeln für den Code

- Spiellogik (`kim/engine`, `leitner/engine`) ist reines Kotlin ohne Android-Imports und vollständig per Unit-Test abgedeckt.
- Jede Runde wird aus einem Seed erzeugt (`kotlin.random.Random(seed)`); gleicher Seed = identische Runde.
- Zeitmessung über Coroutines im ViewModel, nicht im Composable; Timer pausieren, wenn die App in den Hintergrund geht.
- Alle Texte in `strings.xml` (Deutsch). Keine Werbung, kein Netzwerkzugriff, keine Berechtigungen außer Benachrichtigungen.
- Touch-Ziele mindestens 48 dp, Kontrast mindestens 4,5 : 1.

1 · Ziel und Aufbau der App

Die App ist ein Gedächtnistrainer, der auch nach Wochen noch fordert. Mehrere Schwierigkeitsparameter steigen gleichzeitig, und ein adaptiver Modus hält den Spieler bei etwa 70–80 % Trefferquote.

Teil	Inhalt
Start	Heutiges Training (fällige Karten + Kim-Runde + tägliche Challenge), Serie in Tagen, Kacheln zu beiden Spielen, untere Navigation (Start, Statistik, Stapel, Profil).
Kim's Game	Einprägen → Ablenkung → Abfrage → Ergebnis. 5 Modi, 6 Stufen, adaptiver Modus, tägliche Challenge, Fotomodus.
Lernkartei	Stapel mit Karten in Fächern; Schwierigkeit Normal / Streng / Hart; CSV-Import; Erinnerungen.
Statistik	Verlaufskurven, Fehleranalyse nach Änderungsart, Fehler-Heatmap nach Rasterposition, Bestwerte, Serie.

Ablauf einer Kim-Runde

1. **Einprägen** – Raster mit Objekten für eine feste Zeit, Countdown sichtbar.
2. **Pause / Ablenkung** – Raster verschwindet; je nach Stufe leerer Bildschirm oder Ablenkungsaufgabe.
3. **Abfrage** – je nach Modus fehlende Objekte wählen, Änderungen antippen oder Positionen rekonstruieren; Antwortzeit läuft.
4. **Ergebnis** – Punkte, Trefferquote, Vorher/Antwort nebeneinander, Liste aller Änderungen, Aufstiegsfortschritt.

2 · Kim's Game – Regeln und Zahlen

Stufen

Stufe	Objekte	Einprägen	Änderungen	Ähnlichkeit	Pause / Ablenkung	Zusatzregel	Faktor S
1 Rekrut	8	30 s	1 entfernt	gering	3 s leer	–	1,0
2 Soldat	12	25 s	2 entfernt	gering–mittel	5 s leer	Antwortzeit 30 s	1,3
3 Späher	16	20 s	3 (entfernt/vertauscht)	mittel	10 s Kopfrechnen	Fehlklicks kosten Punkte	1,6
4 Scharfschütze	20	15 s	4 (+ Farbe/Drehung)	hoch	20 s Ablenkung	Anzahl Änderungen unbekannt	2,0
5 Elite	25	10 s	5–6 gemischt	sehr hoch	30 s Doppelaufgabe	Positionen exakt	2,5
6 Meister	30–40	8 s	0–8 (auch keine)	maximal + Fake	60 s Verzögerung + Ablenkung	1 Fehler = Runde verloren	3,0

Ähnlichkeit: Anteil der Objekte, die aus derselben Ähnlichkeitsgruppe gezogen werden – gering 0 %, gering–mittel 15 %, mittel 35 %, hoch 60 %, sehr hoch 80 %, maximal 100 %. „Fake" = eine Änderung, die beim Einprägen schon so war (z. B. ein bereits gedrehtes Objekt), um Raten zu bestrafen.

Auf- und Abstieg

- Stufe geschafft: 5 Runden in Folge mit mindestens 85 % Trefferquote. Die nächste Stufe wird freigeschaltet.
- Abstieg: unter 50 % in 3 Runden hintereinander → App schlägt Abstieg vor (Dialog, nicht automatisch).
- Trefferquote = T / (T + F + Ü).

Adaptiver Modus

- Schwierigkeitswert d von 1,0 bis 10,0. Alle Parameter werden aus der Stufentabelle linear interpoliert: Position p = 1 + (d − 1) × 5 / 9 (also d = 1 → Stufe 1, d = 10 → Stufe 6).
- Treppenregel nach jeder Runde: zwei Runden in Folge ≥ 80 % → d + 0,3; eine Runde < 60 % → d − 0,5; Grenzen 1,0 und 10,0.

Ablenkungsaufgaben – nie zweimal dieselbe

Jede Ablenkungsaufgabe wird neu erzeugt und darf über die gesamte Nutzungsdauer nur **ein einziges Mal** vorkommen. Dazu speichert die App jede gezeigte Aufgabe als Schlüssel in Room und verwirft beim Erzeugen jede Aufgabe, deren Schlüssel schon existiert.

Typ	Aufbau einer Aufgabe	Schlüssel	Mögliche Aufgaben
Kopfrechnen	a ∘ b ∘ c mit a, b, c aus 2–99, ∘ aus + − ×; × nur mit einem Faktor ≤ 12; Ergebnis 0–200.	`calc:47+38-19`	mehrere 100 000
Farbwörter (Stroop)	Folge von 4 Farbwörtern, jedes in einer anderen Schriftfarbe als sein Wort; 6 Farben. Alle 4 nacheinander beantworten.	`stroop:RT-BL-GR-GE…`	über 400 000
Rückwärtszählen	Start 100–999, Schrittweite 6, 7, 8 oder 9, fünf Schritte nennen.	`count:731-7`	3 600
Reaktion	Kreis antippen, sobald er grün wird; Zeitpunkt zufällig. Kein Inhalt, der sich merken lässt.	–	–

```
@Entity(primaryKeys = ["type", "key"])
data class UsedDistraction(val type: String, val key: String, val usedAt: Long)

fun next(type): Task {
    repeat(200) { val t = generate(type, rng); if (!dao.exists(type, t.key)) { dao.insert(t); return t }
}
    widenRange(type)   // z. B. Kopfrechnen: Zahlen bis 199 oder 3 Operationen
    return next(type)
}
```

Bei 3 Runden am Tag mit je 5 Rechenaufgaben sind das rund 5 500 Aufgaben pro Jahr – der Vorrat reicht viele Jahre. Wird das Rückwärtszählen knapp (mehr als 80 % verbraucht), erweitert die App automatisch auf Startwerte bis 9 999. Die Room-Datenbank wird per Android Auto Backup gesichert, damit der Verlauf auch nach einem Handywechsel erhalten bleibt. Die Ablenkung zählt nicht zur Punktzahl, die Ergebnisse werden aber in der Statistik gespeichert.

Spielmodi

Modus	Abfrage	Trainiert
Was fehlt?	Objekte verschwinden; Auswahl aus Liste mit Ablenkern (doppelt so viele Optionen wie fehlende Objekte).	Wiedererkennen
Was hat sich verändert?	Objekte vertauscht, umgefärbt, gedreht oder ersetzt; veränderte Felder antippen.	Detailwahrnehmung
Rekonstruktion	Leeres Raster; Objekte per Drag-and-Drop an Originalposition ziehen.	Räumliches Gedächtnis
Freier Abruf	Ohne Auswahlliste alle Objekte nennen (Tippen mit Autovervollständigung aus dem Katalog).	Aktiver Abruf
Sequenz	Objekte erscheinen nacheinander; Reihenfolge wiedergeben, ab Stufe 4 rückwärts.	Arbeitsgedächtnis

Sondermodi: **Tägliche Challenge** (Seed = Datum als `yyyyMMdd`, eine Runde pro Tag, adaptiver Wert des Spielers) und **Fotomodus** (eigene Fotos statt Icons, spätere Version).

Punkte

```
Punkte = round( (10·T − 15·F − 5·Ü) · S · (1 + 0,2 · tRest / tMax) )
T = Treffer, F = Fehlklicks, Ü = übersehene Änderungen,
S = Stufenfaktor (Tabelle oben), tRest/tMax = übrige Antwortzeit.
Negative Werte werden als 0 gespeichert.
```

Beispiel aus dem Ergebnis-Bildschirm: T = 3, F = 1, Ü = 1, S = 2,0, 40 % Restzeit → (30 − 15 − 5) · 2,0 · 1,08 ≈ 22 Punkte.

3 · Lernkartei nach dem Leitner-System

Jede Karte liegt in einem Fach. Richtig → ein Fach weiter, längeres Intervall. Falsch → zurück nach Regel des Modus. Pro Tag werden nur fällige Karten abgefragt.

Modus	Fächer	Intervalle (Tage)	Bei Fehler	Antwortform
Normal	5	1 · 2 · 4 · 8 · 16	ein Fach zurück	aufdecken + selbst bewerten
Streng (Standard)	5	1 · 3 · 7 · 14 · 30	zurück in Fach 1	aufdecken + selbst bewerten, beide Richtungen
Hart	7	1 · 2 · 4 · 8 · 16 · 32 · 64	zurück in Fach 1	eintippen, 15 s Limit, exakt

Algorithmus

```
fällig(karte)   = heute >= karte.nextDue
richtig         → fach = min(fach + 1, letztesFach); nextDue = heute + intervall[fach]
falsch Normal   → fach = max(fach - 1, 1);           nextDue = morgen
falsch Streng/Hart → fach = 1;                       nextDue = morgen
im letzten Fach richtig → Karte gilt als gelernt, bleibt aber mit
                          letztem Intervall in der Wiederholung
Hart: richtig nur wenn antwort.trim().lowercase() == lösung.trim().lowercase()
      Zeitlimit abgelaufen = falsch
Beide Richtungen (Streng): jede Karte hat 2 Einträge (vorwärts/rückwärts)
      mit eigenem Fach
```

Wechselt ein Stapel von 5 auf 7 Fächer (Hart), behalten Karten ihr Fach; beim Wechsel von 7 auf 5 werden Fach 6 und 7 zu Fach 5.

Funktionen

- Stapel anlegen, umbenennen, löschen; Karten mit Text und optional Bild.
- CSV-Import im Format `Vorderseite;Rückseite`, UTF-8, erste Zeile darf Kopfzeile sein; Duplikate werden übersprungen.
- Fächeransicht als Säulen mit Kartenanzahl je Fach (siehe Bildschirm 7).
- Tägliche Erinnerung per WorkManager zur einstellbaren Uhrzeit, nur wenn Karten fällig sind.
- Gemeinsames Tagestraining mit Kim's Game und gemeinsamer Serie (ein Tag zählt, wenn alle fälligen Karten und eine Kim-Runde erledigt sind).

4 · Technik-Stack und Architektur

Bereich	Wahl	Grund
Sprache	Kotlin	Android-Standard, Coroutines für Timer
UI	Jetpack Compose + Material 3	Raster, Animationen, Drag-and-Drop ohne XML
Architektur	MVVM, ViewModel + StateFlow	Zustand überlebt Drehung, testbar
Navigation	Navigation Compose	ein Graph für beide Spiele
Datenbank	Room	Runden, Karten, Verlauf lokal
Einstellungen	DataStore	Stufe, Modus, Sound, Erinnerungszeit
DI	Hilt	mehrere Repositories
Hintergrund	WorkManager	Erinnerungen
Diagramme	Vico	Verlaufskurven
Build	Gradle Kotlin DSL, Version Catalog, minSdk 26, targetSdk aktuell	breite Geräteabdeckung

Paketstruktur

```
app/src/main/java/de/max/gedaechtnis/
  core/        Theme (Farben, Typo), gemeinsame Komponenten, Navigation
  kim/engine/  RoundGenerator, ChangeGenerator, Scorer, Difficulty   (reines Kotlin)
  kim/ui/      LevelScreen, MemorizeScreen, DistractScreen,
               RecallScreen, ResultScreen + ViewModels
  leitner/engine/  Scheduler (Fälligkeit, Fach verschieben), AnswerCheck
  leitner/ui/  DeckScreen, ReviewScreen, CardEditor, CsvImport
  data/        Room (Entities, DAOs), Repositories, ObjectCatalog
  stats/       Auswertung, Diagramme
  home/        HomeScreen (Tagestraining, Serie, Kacheln)
app/src/main/assets/
  balancing.json   alle Zahlen aus Kapitel 2 und 3
  catalog.json     Objekt-Katalog (Kapitel 5)
```

5 · Datenmodell und Objekt-Katalog

```
// Kim's Game
data class GameObject(val id: String, val category: String, val baseObject: String,
    val variant: String, val color: ObjColor, val iconRes: String, val similarityGroup: String)
data class RoundConfig(val mode: Mode, val level: Int?, val difficulty: Double?,
    val objectCount: Int, val columns: Int, val memorizeSec: Int, val pauseSec: Int,
    val distraction: Distraction?, val changeCount: IntRange, val changeTypes: Set<ChangeType>,
    val showChangeCount: Boolean, val answerSec: Int?, val seed: Long)
enum class ChangeType { REMOVED, REPLACED, SWAPPED, RECOLORED, ROTATED, FAKE }
data class Change(val type: ChangeType, val position: Int, val before: GameObject?, val after:
GameObject?)
@Entity data class RoundResult(@PrimaryKey(autoGenerate = true) val id: Long = 0,
    val date: Long, val configJson: String, val hits: Int, val falseClicks: Int, val missed: Int,
    val reactionMs: Long, val points: Int, val changesJson: String)

// Lernkartei
@Entity data class Deck(@PrimaryKey(autoGenerate = true) val id: Long = 0, val name: String,
    val mode: LeitnerMode /* NORMAL, STRICT, HARD */)
@Entity data class Card(@PrimaryKey(autoGenerate = true) val id: Long = 0, val deckId: Long,
    val front: String, val back: String, val imageUri: String?,
    val box: Int, val nextDue: Long /* epochDay */, val direction: Direction)
@Entity data class ReviewLog(@PrimaryKey(autoGenerate = true) val id: Long = 0, val cardId: Long,
    val date: Long, val correct: Boolean, val answerMs: Long, val boxBefore: Int, val boxAfter: Int)
```

Objekt-Katalog

- Ziel Version 1: 300 Objekte in 15 Kategorien (Werkzeug, Elektronikbauteile, Fahrzeuge, Tiere, Lebensmittel, Symbole, Formen …), je Grundobjekt 3–5 Varianten (Farbe, Größe, Detail).
- Für Phase 1 reichen 60 Objekte in 5 Kategorien.
- Icons als Vector Drawables mit einheitlicher Strichstärke (1,7 von 24), eingefärbt zur Laufzeit mit den 4 Objektfarben aus Kapitel 6. Quellen: Material Symbols (Apache 2.0) oder eigene SVGs.
- Die Ähnlichkeitsgruppe steuert die Schwierigkeit: auf hohen Stufen zieht der Generator bevorzugt aus derselben Gruppe.

```
// catalog.json (Auszug)
[{ "id": "key_amber", "category": "werkzeug", "base": "key", "variant": "standard",
   "color": "amber", "icon": "ic_key", "group": "key" },
 { "id": "key_cyan",  "category": "werkzeug", "base": "key", "variant": "standard",
   "color": "cyan",  "icon": "ic_key", "group": "key" }]
```

Statistik

- Trefferquote und erreichte Stufe pro Tag als Verlaufskurve.
- Fehleranalyse nach Änderungsart (welche Art wird am häufigsten übersehen).
- Fehler-Heatmap nach Rasterposition (blinde Flecken).
- Serie, Bestwerte je Modus, Leitner: Karten je Fach im Zeitverlauf.

6 · Design-System

Dunkles, technisches Design. Kim's Game hat Amber als Akzent, die Lernkartei Cyan. Nur ein dunkles Theme (kein Hell-Modus in Version 1).

Farben

Token	Hex	Verwendung
Background	`#0E1115`	Bildschirmhintergrund
Surface	`#161B21`	Karten, Rasterfelder, Kacheln
SurfaceNav	`#12161B`	Untere Navigation, Eingabefelder
Outline	`#2B343E`	Rahmen 1 dp, Trennlinien, leere Fortschrittsbalken
TextPrimary	`#E9EEF2`	Haupttext, helle Schaltfläche
TextSecondary	`#A3AEB9`	Hinweise, Labels, Parameter
KimAccent	`#F5B041`	Kim: Hauptbutton, Auswahl, Timer, Serie; Text darauf `#14100A`
LeitnerAccent	`#5BC8DC`	Lernkartei und Ablenkung; Text darauf `#0B1A1E`
LeitnerBoxMuted	`#2F5E68`	Säulen der Fächer 2 bis n
Success	`#6BD49A`	richtig / erkannt (Rahmen durchgezogen)
Error	`#F07167`	falsch / Fehlklick (Rahmen gestrichelt), Timer unter 5 s

Objektfarben im Raster: Amber #F5B041, Cyan #5BC8DC, Weiß #E9EEF2, Koralle #F07167. Übersehene Änderungen: Amber, Rahmen gepunktet. Zustände unterscheiden sich immer auch über Rahmenart oder Text, nie nur über Farbe.

Typografie (Google Fonts, als Font-Ressourcen einbinden)

Rolle	Schrift	Größen (sp)
Überschriften, Kachel-Titel, Stufennamen	Space Grotesk Bold	26 · 24 · 20 · 19 · 17 · 16
Fließtext, Buttons, Labels	IBM Plex Sans 400/500/600	16 · 15 · 14 · 13 · 12
Zahlen, Timer, Punkte, Antworteingabe	IBM Plex Mono Medium	44 · 36 · 34 · 24 · 22 · 18
Abschnittslabels (MODUS, STUFE, SPIELE)	IBM Plex Sans 500, Großbuchstaben, Laufweite +0,08 em	13

Formen und Abstände

Element	Maße
Bildschirmrand	20 dp (Raster 16 dp)
Abstände	4 / 8 / 12 / 16 dp; Rasterfelder 8 dp Abstand
Radien	große Karten 18–22 dp, Buttons 12–14 dp, Rasterfelder 14 dp, Chips voll gerundet (20 dp)
Hauptbutton	Höhe 52 dp, volle Breite, Akzentfarbe, Text 16 sp SemiBold
Nebenbutton	Höhe 48–52 dp, transparent mit 1 dp Outline
Icon-Buttons	44 × 44 dp, Radius 12 dp
Rasterfeld Kim	4 Spalten, Höhe 92 dp bei 20 Objekten; Icon 40 dp. Bei 30–40 Objekten 6 Spalten, Feld mind. 48 dp
Ausgewähltes Element	2 dp Rahmen in Akzentfarbe + Fläche Akzent mit 10 % Deckkraft

Compose-Theme

```
object AppColors {
    val Background      = Color(0xFF0E1115)
    val Surface         = Color(0xFF161B21)
    val SurfaceNav      = Color(0xFF12161B)
    val Outline         = Color(0xFF2B343E)
    val TextPrimary     = Color(0xFFE9EEF2)
    val TextSecondary   = Color(0xFFA3AEB9)
    val KimAccent       = Color(0xFFF5B041)
    val OnKimAccent     = Color(0xFF14100A)
    val LeitnerAccent   = Color(0xFF5BC8DC)
    val OnLeitnerAccent = Color(0xFF0B1A1E)
    val LeitnerBoxMuted = Color(0xFF2F5E68)
    val Success         = Color(0xFF6BD49A)
    val Error           = Color(0xFFF07167)
}
```

7 · Bildschirme

Acht Bildschirme, Vorlage 390 × 844 dp. Texte, Reihenfolge und Zustände wie abgebildet umsetzen.



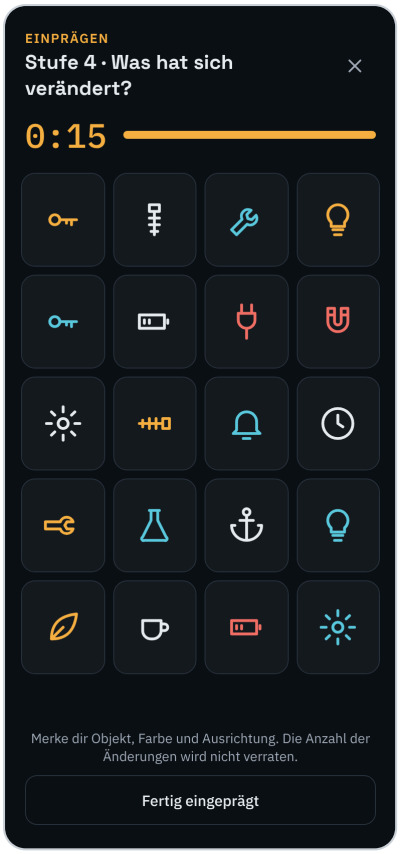
7.1 Start `home/HomeScreen`

- Kopf: Datum (TextSecondary 13 sp), Titel „Gedächtnistrainer“, Icon-Button Einstellungen.
- Karte „Heutiges Training“: Checkliste mit Kim-Runde, fälligen Karten, täglicher Challenge; erledigte Punkte grün abgehakt und durchgestrichen; Zähler „x / 3 erledigt“; heller Button „Training fortsetzen“ startet den nächsten offenen Punkt.
- Serie: Zahl in Mono + 7 Kästchen der letzten Tage (gefüllt = erledigt, gestrichelt = heute offen).
- Abschnitt „SPIELE“: zwei Kacheln. Kim: aktuelle Stufe, Fortschrittsbalken bis zum Aufstieg. Lernkartei: Anzahl Stapel, Modus, fällige Karten groß in Cyan.
- Untere Navigation: Start, Statistik, Stapel, Profil.



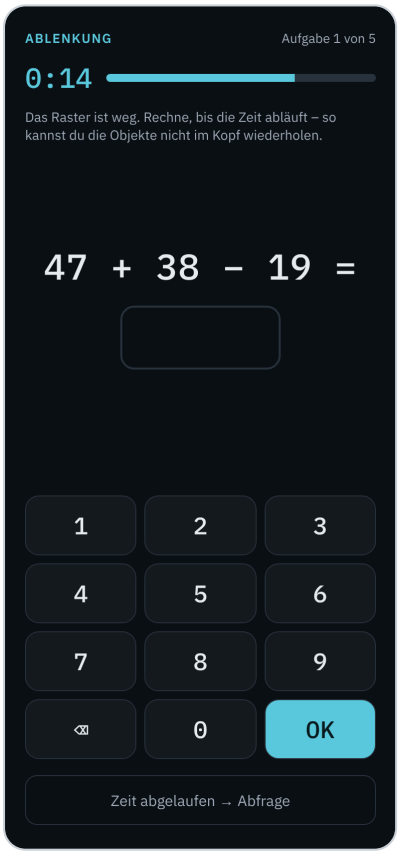
7.2 Kim – Modus und Stufe `kim/ui/LevelScreen`

- Modus-Chips (umbrechend): Was fehlt?, Was hat sich verändert?, Rekonstruktion, Freier Abruf, Sequenz. Auswahl = Amber gefüllt.
- Stufenliste 1–6: Nummer (Mono), Name, Parameterzeile, rechts Status („✓ geschafft“, „3 / 5“, „gesperrt“). Gesperrte Stufen 55 % Deckkraft und nicht wählbar.
- Karte „Adaptiver Modus“ mit Schalter und aktuellem Wert „Schwierigkeit 6,4 von 10,0“. Aktiv → Stufenauswahl wird aufgehoben.
- Hauptbutton „Runde starten · Stufe n“ bzw. „· adaptiv“.



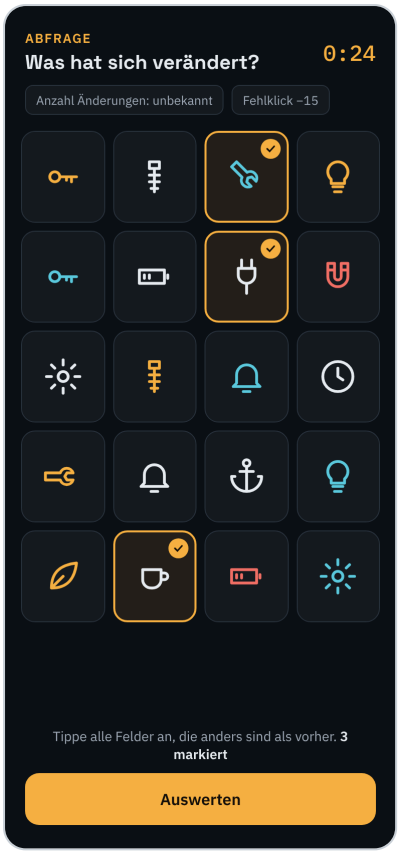
7.3 Kim – Einprägen `kim/ui/MemorizeScreen`

- Label „EINPRÄGEN“ in Amber, Titel „Stufe n · Modusname“, Schließen-Button (Runde abbrechen mit Rückfrage).
- Countdown in Mono 34 sp + Balken, der linear abläuft; letzte 5 s in Error-Rot.
- Raster aus Surface-Feldern mit Objekt-Icons in 4 Objektfarben, teils gedreht (0°, 45°, 90°).
- Hinweistext unten; Nebenbutton „Fertig eingeprägt“ beendet die Phase vorzeitig (Restzeit wird nicht gutgeschrieben).
- Bei Zeitablauf automatisch weiter zur Pause/Ablenkung.



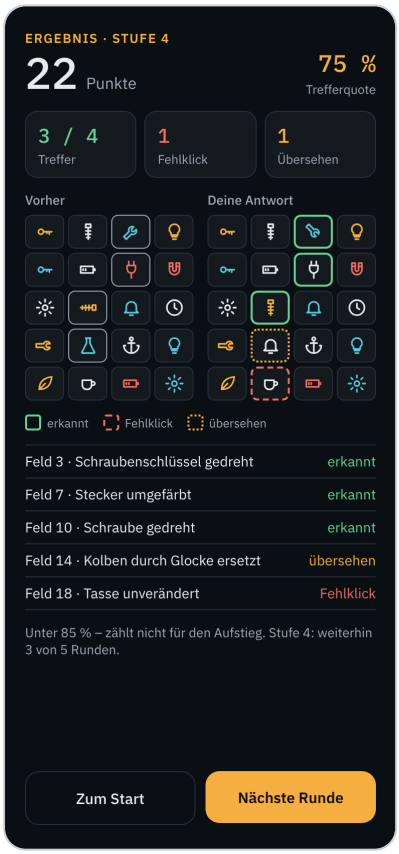
7.4 Kim – Ablenkung `kim/ui/DistractScreen`

- Label „ABLENKUNG“ in Cyan, „Aufgabe x von 5“, Countdown in Cyan mit Balken.
- Erklärtext: Raster ist weg, Rechnen verhindert inneres Wiederholen.
- Aufgabe in Mono 36 sp, Ausgabefeld mit Rahmen (grün bei richtig, rot bei falsch + richtige Lösung).
- Ziffernblock 3 × 4 (1–9, Löschen, 0, OK in Cyan), Tasten 60 dp hoch.
- Nach Bestätigung 0,9 s Feedback, dann nächste Aufgabe. Bei Zeitablauf weiter zur Abfrage.



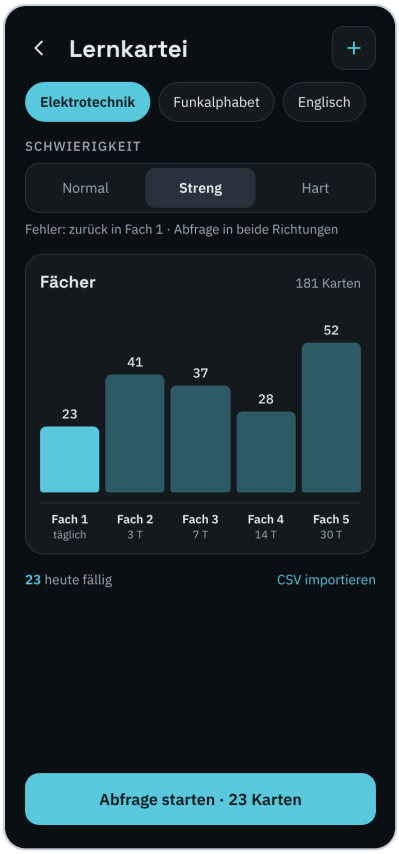
7.5 Kim – Abfrage `kim/ui/RecallScreen`

- Label „ABFRAGE“, Titel des Modus, Antwort-Countdown oben rechts.
- Info-Chips: „Anzahl Änderungen: unbekannt“ (ab Stufe 4, sonst „Änderungen: n“), „Fehlklick –15“.
- Gleiches Raster wie beim Einprägen, mit angewendeten Änderungen. Antippen schaltet Markierung um: Amber-Rahmen 2 dp, Amber-Fläche 10 %, Häkchen-Badge oben rechts.
- Zähler „n markiert“, Hauptbutton „Auswerten“.
- Andere Modi nutzen denselben Kopf; Rekonstruktion: leeres Raster + Objektleiste zum Ziehen.



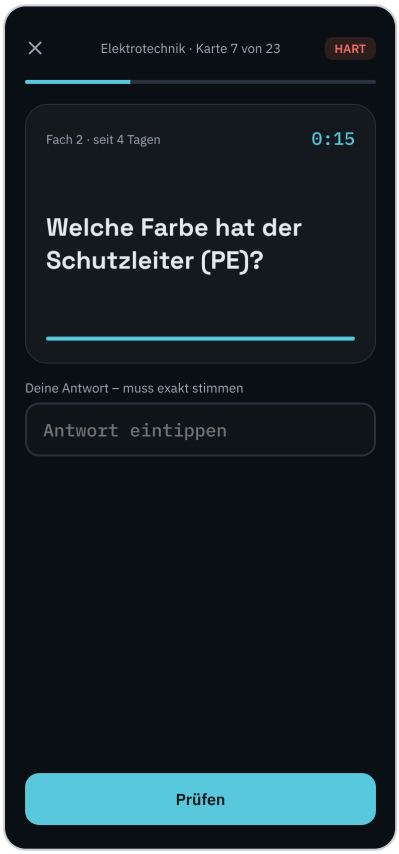
7.6 Kim – Ergebnis `kim/ui/ResultScreen`

- Punkte groß in Mono 44 sp, Trefferquote rechts in Amber.
- Drei Kacheln: Treffer (grün), Fehlklick (rot), Übersehen (Amber).
- Zwei Mini-Raster „Vorher“ und „Deine Antwort“ nebeneinander. Rahmen: erkannt = grün durchgezogen, Fehlklick = rot gestrichelt, übersehen = Amber gepunktet; im Vorher-Raster sind alle Änderungen grau umrandet. Legende darunter.
- Liste aller Änderungen und Fehlklicks mit Feldnummer, Objekt, Art und Ergebnis.
- Satz zum Aufstieg („Unter 85 % – zählt nicht …“). Buttons „Zum Start“ und „Nächste Runde“.



7.7 Lernkartei – Fächer `leitner/ui/DeckScreen`

- Kopf mit Zurück, Titel „Lernkartei“, Plus-Button (Karte hinzufügen).
- Stapel-Chips (Auswahl Cyan gefüllt).
- Segment-Schalter Normal / Streng / Hart mit Erklärzeile zur Fehlerregel.
- Karte „Fächer“: Säulen je Fach, Höhe proportional zur Kartenanzahl (max. 150 dp), Zahl darüber; Fach 1 in Cyan, übrige in LeitnerBoxMuted; darunter „Fach n“ und Intervall („täglich“, „3 T“ …). Bei Hart 7 Säulen.
- „n heute fällig“, Link „CSV importieren“, Hauptbutton „Abfrage starten · n Karten“.



7.8 Lernkartei – Abfrage (Hart) `leitner/ui/ReviewScreen`

- Kopf: Schließen, „Stapel · Karte x von n“, Badge „HART“ in Rot; Fortschrittsbalken.
- Große Karte: „Fach n · seit x Tagen“, 15-s-Countdown (Mono, letzte 5 s rot) mit Balken, Frage in Space Grotesk 25 sp.
- Eingabefeld „Deine Antwort – muss exakt stimmen“, Mono 18 sp, Tastatur öffnet automatisch.
- „Prüfen“ → Rahmen grün/rot, Ergebnisbox („Richtig – weiter in Fach n“ bzw. „Falsch – zurück in Fach 1“ / „Zeit abgelaufen …“) mit richtiger Antwort, Button wird „Nächste Karte“.
- Normal/Streng: statt Eingabe Button „Aufdecken“, danach „Gewusst“ / „Nicht gewusst“.

8 · Bau-Reihenfolge (Phasen für Claude Code)

Jede Phase ist ein eigener Auftrag an Claude Code. Erst weiter, wenn das Abnahmekriterium erfüllt ist.

Phase 1 · Grundgerüst

- Android-Projekt (Compose, Hilt, Room, DataStore, Navigation), Theme aus Kapitel 6, Schriften eingebunden.
- `kim/engine`: RoundGenerator mit Seed, ChangeGenerator (entfernt, vertauscht), Scorer.
- Modus „Was fehlt?“ mit Stufen 1–3; Bildschirme 7.1 (vereinfacht), 7.2, 7.3, 7.5, 7.6.
- 60 Objekte in 5 Kategorien, `balancing.json`.

Fertig, wenn: eine Runde komplett durchläuft und alle Engine-Tests grün sind.

Phase 2 · Volle Schwierigkeit

- Stufen 4–6: Drehung, Umfärben, Ersetzen, Fake-Änderungen, „Anzahl unbekannt“, Ein-Fehler-Regel.
- Ablenkungsaufgaben (Bildschirm 7.4): Kopfrechnen, Stroop, 7er-Reihe, Reaktion.
- Modus „Was hat sich verändert?“, Auf-/Abstiegslogik, Katalog auf 300 Objekte.

Fertig, wenn: alle 6 Stufen spielbar sind und Stufe 6 im Selbsttest unter 40 % Treffer liegt.

Phase 3 · Modi und Statistik

- Rekonstruktion (Drag-and-Drop), Sequenz, Freier Abruf.
- RoundResult in Room, Statistik-Bildschirm mit Verlauf, Fehleranalyse, Heatmap.
- Ergebnis-Bildschirm vollständig wie 7.6.

Fertig, wenn: 7 Tage Verlauf sichtbar sind und die Fehleranalyse stimmt.

Phase 4 · Adaptiv und Feinschliff

- Adaptiver Modus mit Treppenregel, tägliche Challenge (Seed = Datum).
- Balancing nur über `balancing.json`, Sound/Vibration, Timer-Pause im Hintergrund.
- Signierte Release-APK.

Fertig, wenn: die Stufen kalibriert sind und die APK auf dem Handy läuft.

Phase 5 · Lernkartei (Leitner)

- `leitner/engine`: Scheduler und AnswerCheck nach Kapitel 3, mit Unit-Tests (alle drei Modi, Fachwechsel 5 ↔ 7, Zeitlimit).
- Bildschirme 7.7 und 7.8, Karteneditor, CSV-Import, Erinnerungen per WorkManager.
- Gemeinsames Tagestraining und Serie auf dem Startbildschirm (7.1 vollständig).

Fertig, wenn: Karten zum richtigen Tag fällig werden und beide Spiele eine Serie teilen.

9 · Tests und Abnahme

Automatisch (Claude Code muss diese schreiben und ausführen)

- RoundGenerator: richtige Objektanzahl je Stufe, keine doppelten Positionen, gleicher Seed = gleiche Runde, Ähnlichkeitsanteil eingehalten.
- ChangeGenerator: Änderungen nur an gültigen Feldern, Anzahl im Bereich der Stufe, Fake-Änderungen nur ab Stufe 6, „keine Änderung“ möglich in Stufe 6.
- Scorer: Treffer, Fehlklicks, Übersehene korrekt gezählt; Formel inkl. Zeitbonus; nie negativ gespeichert.
- Auf-/Abstieg: 5 × ≥ 85 % schaltet frei, 3 × < 50 % löst Vorschlag aus.
- Adaptiv: Treppenregel und Grenzen 1,0 / 10,0.
- Ablenkung: 10 000 erzeugte Aufgaben je Typ enthalten keinen doppelten Schlüssel; nach Neustart der App wird keine frühere Aufgabe erneut gezeigt; Bereichserweiterung greift, wenn der Vorrat erschöpft ist.
- Leitner-Scheduler: Fälligkeiten für alle Modi, Rückfall-Regeln, Umrechnung 7 → 5 Fächer, Hart-Vergleich (Groß/Klein egal, Leerzeichen am Rand egal, sonst exakt).
- CSV-Import: Semikolon, UTF-8 mit Umlauten, Kopfzeile, Duplikate.
- UI-Tests (Compose Testing): kompletter Kim-Rundenablauf und eine Leitner-Abfrage.

Manuell

- Eine Woche selbst spielen: Stufe 1 ca. 95 %, Stufe 6 unter 40 % Treffer.
- 2–3 andere Personen testen lassen (eigene Gewöhnung an die Objekte verfälscht die Einschätzung).
- Kleines Display (5") und Tablet: bei 40 Objekten jedes Feld mindestens 48 dp.
- App in den Hintergrund während Einprägen: Timer pausiert, Runde wird nicht gewertet, wenn länger als 3 s weg.

> **Definition of Done:** alle Tests grün, keine Lint-Fehler, Bildschirme entsprechen Kapitel 7, alle Zahlen kommen aus `balancing.json`, App läuft ohne Internet.